Politechnika Śląska Katedra Grafiki, Wizji komputerowej i Systemów Cyfrowych				
Rok akademicki	Rodzaj studiów*: SSI/NSI/NSM	Przedmiot: (Języki Asemblerowe/SMIW/BIAI)	Grupa	Sekcja
2025/2026	SSI	JA – PROJ	5	6
Prowadzący przedmiot:	dr inż. Adam Opara		Termin zajęć: (dzień tygodnia godzina)	
Skład sekcji: Email kontaktowy:	Piotr Szlejter piotszl618@student.polsl.pl		Wtorek 11:30-13:00	

Raport

Temat Projektu: Symulator Conway Game of life – symulator automatu komórkowego

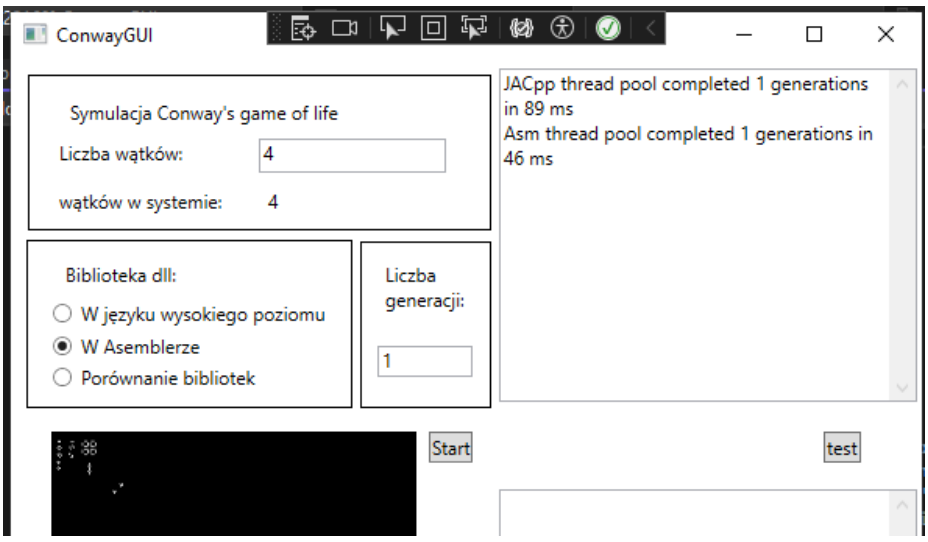
Opis celu implementacji algorytmu:

Celem implementacji algorytmu było zbadania zwiększenia wydajności programu przetwarzającego duże ilości danych – w tym wypadku duże tablice komórek automatu Conway game of life, przy zastosowaniu instrukcji wektorowych.

Opis działania programu:

Program otwiera się z prostym interfejsem graficznym stworzonym w c#, w którym użytkownik wybiera liczbę wątków z którą ma się uruchomić algorytm, liczbę pokoleń gry Conway do obliczenia oraz opcję czy uruchomić bibliotekę asemblerową lub w języku wysokiego poziomu. Po wybraniu opcji „Start” program wczytuję bitmapę monochromatyczną z katalogu w którym znajduje się plik „.exe” ładuje ją do pamięci i uruchamia zadaną liczbę wątków dzieląc wiersze bitmapy pomiędzy wątki realizujące algorytm.
Po zakończeniu w interfejsie graficznym pojawia się obliczony czas wykonania zadania wraz z podglądem ostatniego obliczonego pokolenia.

Wygląd GUI:



Krytyczny fragment asemblerowego kodu:

```
VectorLoop:
    cmp r11, rax
    jg CheckRemaining

    ; above
    vmovdqu ymm0, ymmword ptr [rsi + r11 - 1]
    vmovdqu ymm1, ymmword ptr [rsi + r11]
    vmovdqu ymm2, ymmword ptr [rsi + r11 + 1]
    vpaddb  ymm0, ymm0, ymm1
    vpaddb  ymm0, ymm0, ymm2
    ; middle
    vmovdqu ymm3, ymmword ptr [rcx + r11 - 1]
    vmovdqu ymm4, ymmword ptr [rcx + r11 + 1]
    vpaddb  ymm0, ymm0, ymm3
    vpaddb  ymm0, ymm0, ymm4
    ; below
    vmovdqu ymm5, ymmword ptr [rdi + r11 - 1]
    vmovdqu ymm1, ymmword ptr [rdi + r11]
    vmovdqu ymm2, ymmword ptr [rdi + r11 + 1]
    vpaddb  ymm0, ymm0, ymm5
    vpaddb  ymm0, ymm0, ymm1
    vpaddb  ymm0, ymm0, ymm2

    ; Conway rules
    vmovdqu ymm8, ymmword ptr [rcx + r11]
    vpcmpeqb ymm1, ymm0, ymm13              ; Count == 2
    vpcmpeqb ymm2, ymm0, ymm14              ; Count == 3
    vpand ymm1, ymm1, ymm8                  ; (Count == 2) & Alive
    vpor  ymm1, ymm1, ymm2                  ; OR (Count == 3)
    vpand ymm1, ymm1, ymm15                 ; turn to 0000 0001b
    vmovdqu ymmword ptr [rdx + r11], ymm1   ; Store

    add r11, 32
    jmp VectorLoop
```

Opis fragmentu:

W tym fragmencie znajduje się główna pętla funkcji w bibliotece asemblerowej. Na samym początku sprawdzany jest czy wykroczyliśmy poza koniec wiersza pomniejszony o 32 bajty, ponieważ w rejestrze r11 znajduje się aktualny indeks kolumny – pozycja w wierszu w której jest program, a rejestr rax zawiera ten limit czyli szerokość wiersza pomniejszona o 33 bajty (32 bajty + 1 bajt graniczny wokół bitmapy którego nie przetwarzamy). Po skoku program wyznaczy wskaźnik na ostatnie 32 bajty dla którego ponownie wykona się główna pętla, wyrównując obliczone komórki do końca, ale możliwie powtarzając obliczenia dla tej części ostatnich 32 bajtów która już były obliczona.

W dalszej części następuje zliczanie żywych sąsiadów 32 aktualnych komórek, dla przykładu po załadowaniu do rejestru ymm0 wartości [rsi + r11 - 1] , w rezultacie otrzymamy liczbę żywych sąsiadów z lewej górnej strony, dla komórek o 32 adresach do których prowadzą adresy [rcx + r11] , dlatego też nie dodajemy ich do obliczonej liczby żywych sąsaidów. Rejestr rsi przechowuje adres wierszu powyżej, rejestr rcx przechowuje bieżący wiersz, rejestr rdi przechowuje wiersz poniżej.

Następnie wykonuje się sprawdzenie warunków Conway Game of life warunki te to:

- Martwa komórka, która ma dokładnie 3 żywych sąsiadów, staje się żywa.

- Żywa komórka z 2 albo 3 żywymi sąsiadami pozostaje nadal żywa, przy innej liczbie sąsiadów umiera.

W rejestrze ymm8 znajduje się aktualny stan 32 komórek.

W rejestrze ymm1 jest najpierw informacja o tym czy jest 2 żywych sąsiadów, a następnie informacja o tym czy komórka jest żywa w następnym pokoleniu.

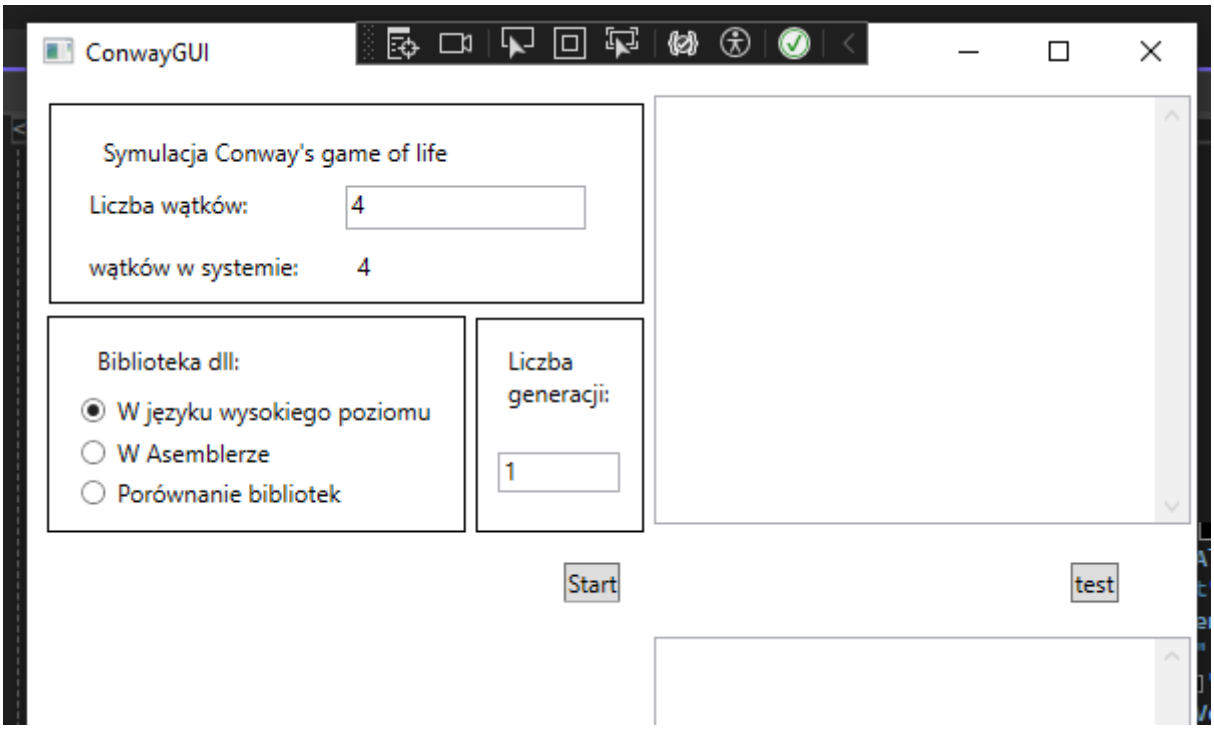
W rejestrze ymm2 znajduje się informacja o tym czy jest 3 żywych sąsiadów.

W rejestrach ymm13, ymm14, ymm15, są stałe 2, 3, 1, stałe te są w każdym z 32 bajtów w rejestrze.

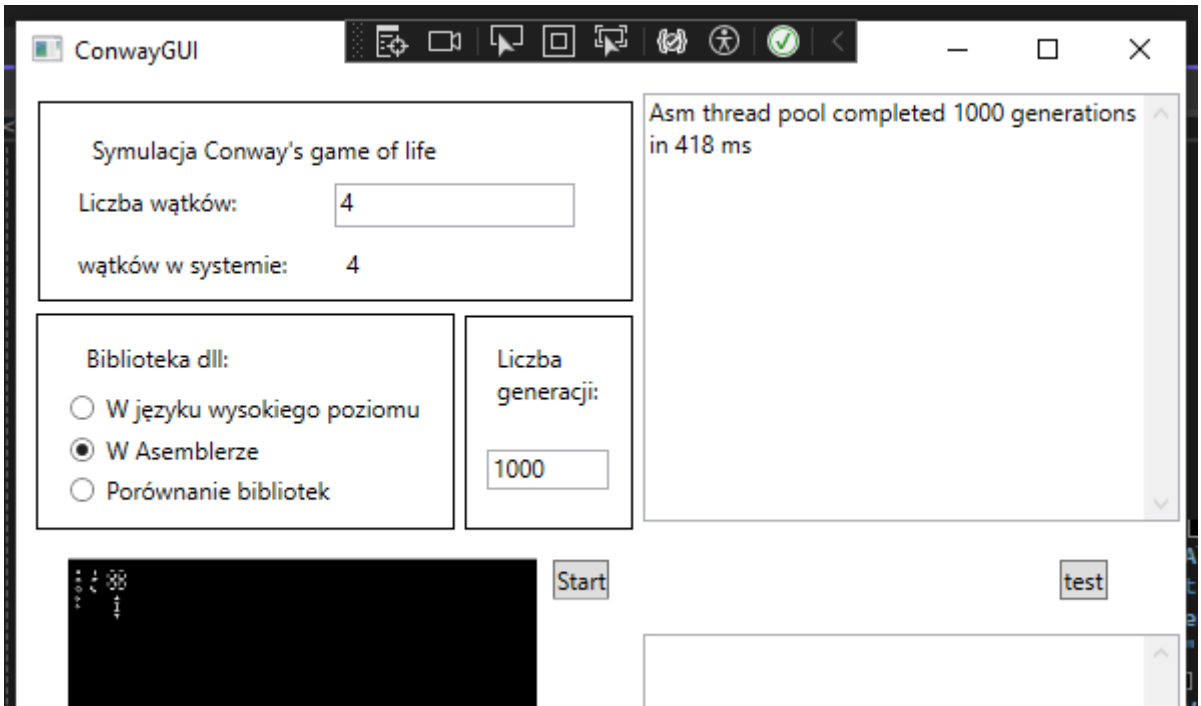
Dzięki operacji „vpcmpeqb” możemy sprawdzić czy każda z 32 ilości sąsiadów jest równa 2 lub 3 z rejestrów ymm13, ymm14. Dzięki operacji „vpand”, „vpor” możemy połączyć wyniki sprawdzenia warunków ilości sąsiadów z tym czy komórka jest żywa. Na koniec dzięki operacji „vpand” stan każdego z 32 bajtów jest konwertowany z 11111111 lub 00000000 które otrzymujemy np. po instrukcji „vpcmpeqb” do postaci 00000001 lub 00000000, czyli postaci w której przechowujemy stan komórki w bitmapie. Zastosowane instrukcje „vpand”, „vpor” zastępują konstrukcję if else z języku wysokiego poziomu, która powodowałaby możliwe błędne przewidywanie skoku - branch prediction, przez co program wykonywałby się wolniej.

Na koniec licznik pozycji w wierszu jest inkrementowany o kolejne 32 bajty i następuje skok do pętli głównej.

Program przed wykonaniem:



Program po wykonaniu:



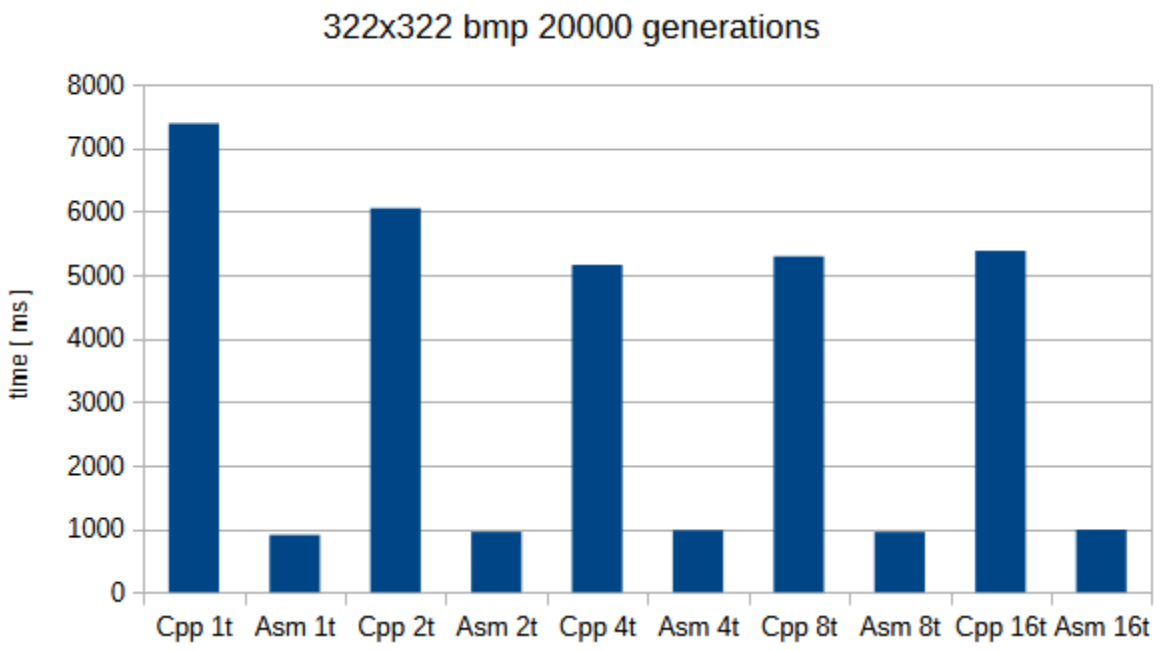
Wyniki czasowe:

Ponieważ przetwarzamy 32 komórki jednocześnie, teoretycznie program może działać 32x szybciej.

Aby nie kopiować wynikowych bitmap, co obliczone pokolenie, algorytm podmienia 2 tablice na zmianę traktując je na zmianę jako input i output.

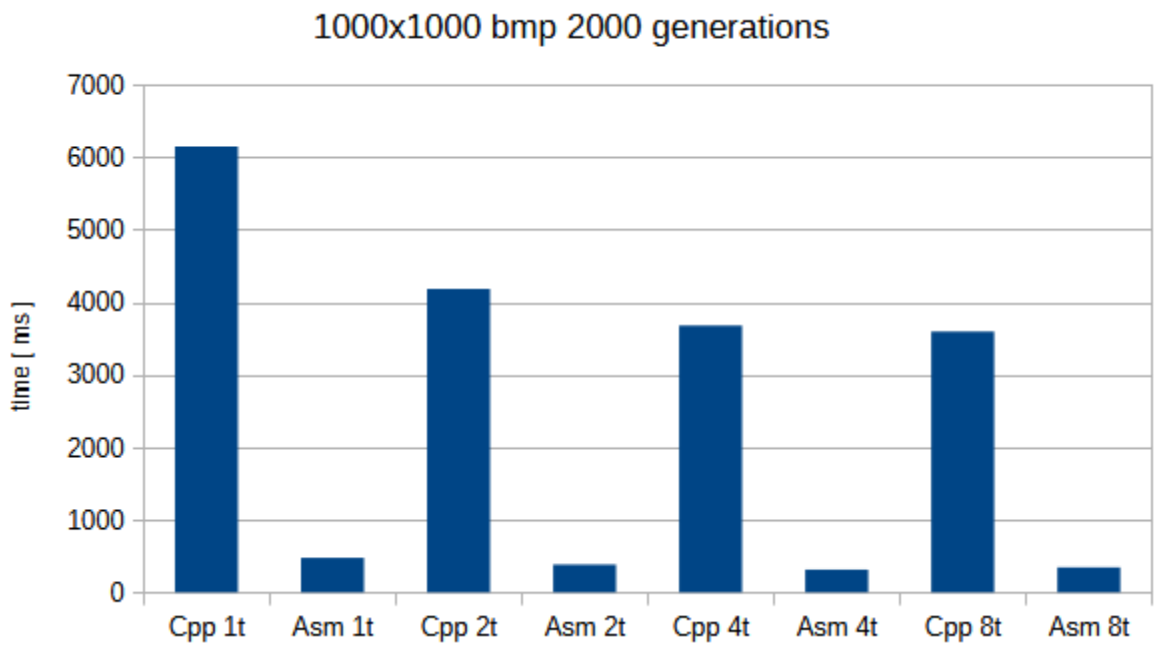
Program został przetestowany przez uruchomienie 5 razy na zmianę biblioteki w C++ i biblioteki w Asm. Z bitmapą o rozmiarze 322x322 piksele, z ustawieniem 20000 pokoleń. Zostało to wykonane dla ustawienia 1, 2, 4, 8, 16 wątków.

Wyniki testów 322x322 px, 20000 pokoleń:



Przez to że wyniki poszczególnej ilości wątków były bardzo zbliżone co mogło być wywołane małą ilością obliczeń na 1 wątek w porównaniu do czasu zajmowanego przez uruchomienie zadania w wątku, testy zostały powtórzone z bitmapą 1000x1000 pikseli oraz 2000 pokoleń, dzięki czemu różnica w działaniu pomiędzy różnymi ilościami wątków stała się zauważalna. Wyniki testów:

Wyniki testów 1000x1000 px, 2000 pokoleń:



Wnioski:

- Wyniki potwierdzają, że na 2-rdzeniowym procesorze z Hyper-Threadingiem optymalną wydajność zapewniają 4 wątki, przy czym wyraźne korzyści ze skalowania pojawiają się dopiero przy większych obrazach, gdzie czas właściwych obliczeń zaczyna dominować nad narzutem synchronizacji.

Literatura:

- https://learn.microsoft.com/en-us/cpp/build/walkthrough-creating-and-using-a-dynamic-link-library-cpp?view=msvc-170

- https://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html

- https://learn.microsoft.com/en-us/dotnet/api/system.drawing.bitmap.lockbits?view=windowsdesktop-10.0

- https://learn.microsoft.com/en-us/dotnet/api/system.drawing.imaging.bitmapdata.scan0?view=windowsdesktop-10.0

- https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.copy?view=net-10.0

- https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/unsafe-code

- https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life